Corporate App (versión final)

Desarrollo de Interfaces (0488) · 2º DAM Nebrija · Jorge González Gómez · Ionic 9 + Angular 22 standalone · actualizada el 26-10-2026

> La primera entrega llegaba hasta el punto 9 (+ campo vendedor). La he rehecho desde cero a mano (25-10-2026) y he añadido lo que faltaba: **geolocalización** con la distancia a la oficina (fórmula de Haversine) en Nosotros y un **formulario de contacto** que guarda los mensajes en el dispositivo con Capacitor Preferences.

Qué tiene la app

Página	Qué hace	Archivos
Home	Menú horizontal (`ion-segment` + `routerLink`) a Productos, Nosotros y Contacto	`pages/home/`
Productos	Grid (`ion-grid`) con 3 productos leídos de un JSON local: id, nombre, **vendedor**, unidades, precio y foto	`pages/productos/`, `services/products.ts`, `models/product.interface.ts`, `assets/data/products.json`
Nosotros	Pide la posición del usuario (`@capacitor/geolocation`) y calcula los km hasta la oficina con Haversine	`pages/nosotros/`, `services/geolocation.ts`
Contacto	Correo + mensaje con `[(ngModel)]`; se guardan con `@capacitor/preferences` y siguen ahí al recargar	`pages/contacto/`, `services/messages.ts`

Inyección de dependencias: los tres servicios llevan `@Service()` (una sola instancia para toda la app) y las páginas los piden con `inject()`. Las rutas usan `loadComponent` (lazy loading) y el estado de cada página va en `signal()` (Angular 22 sin zone.js).

Lo más interesante del código

```
// services/geolocation.ts — distancia en km entre dos coordenadas (Haversine)
const R = 6371;                                          // radio de la Tierra en km
const dLat = (lat2 - lat1) * Math.PI / 180;              // grados → radianes
const dLon = (lon2 - lon1) * Math.PI / 180;
const a = Math.sin(dLat/2)**2 + Math.cos(lat1*Math.PI/180) * Math.cos(lat2*Math.PI/180) *
Math.sin(dLon/2)**2;
return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));   // arco = radio × ángulo
```

```
// services/messages.ts — Preferences solo guarda texto: JSON.stringify / JSON.parse
async guardar(correo: string, mensaje: string) {
  const lista = await this.cargar();
  lista.push({ correo, mensaje, fecha: new Date().toLocaleString('es-ES') });
  await Preferences.set({ key: this.clave, value: JSON.stringify(lista) });
}
```

En la página Nosotros hay tres estados (`@if` distancia / `@else if` error / `@else` calculando): si el usuario deniega el permiso de ubicación, se muestra un mensaje en vez de romper la app.

Comprobaciones y pendiente

✔ `ng build` sin errores · ✔ probada página a página en el navegador (menú, productos, geolocalización y mensajes que siguen guardados al recargar)
Pendiente: los puntos de Android (generar la app nativa con Capacitor).

Código y demo

Carpeta en GitHub: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/02-CorporateApp/corporateApp
Commit de la versión final (25-10-2026): cebe308 · Corporate App rehecha a mano con geolocalización y contacto
Demo en Vercel: desarrollo-interfaces-dam-2.vercel.app
Para probarla en local: `cd corporateApp` → `npm install` → `ionic serve` → `http://localhost:8100`